Real-Time Detection & Mitigation of Data Poisoning Attacks on Continuously Retrained Cloud IDS

Project Implementation State, Architecture, Industry 5.0 Agents & Next-Steps Guide

Institution:	Dawood University of Engineering & Technology, Karachi	**Supervisor:**	Sir Arfan Naich
Department:	Department of Cybersecurity (BS Cybersecurity, 23 Fall)	**Release Version:**	**Release v2.0.0** (Git Tagged)
Team Members:	Sofia Naseem (23F-BSCY-40), Sobia Khan (23F-BSCY-54), Hiba Siddiqui (23F-BSCY-56)	**Status:**	**Phase 1-8 Complete + 2 Agents Active**

1. Executive Overview & Scope Realignment

Per explicit supervisor guidance from Sir Arfan Naich, all legacy ransomware detection, file-system entropy monitors, and dummy sandbox encryption modules were **completely removed from the repository scope**. The project is strictly re-aligned on **Adversarial Machine Learning (AML)**: protecting continuously retrained Cloud Intrusion Detection Systems (IDS) against stealth training data poisoning attacks.

Core Threat Model: Threat actors submit subtly manipulated network flow telemetry (5%, 10%, 20% poisoning budgets) such as stealth label flipping (inverting malicious attacks into benign labels) or crafted sample injection to blind the victim IDS model during periodic retraining. Our system acts as an upstream multi-layer security firewall and governance gate.

2. System Architecture & Implemented Components

- **Anti-Leakage Data Pipeline:** Generates/processes 10,000 realistic network flow telemetry records (DoS, port scans, brute force, and benign RPCs). Features are normalized via StandardScaler fitted strictly on training data with zero leakage. Provenance tracked via SHA-256 hashes in split_manifest.json.
- **Controlled Attack Harness:** Implements reproducible label-flipping and crafted sample-injection attacks at 5%, 10%, and 20% budgets. Ground-truth poison labels are preserved strictly for evaluation metrics and never exposed to defense layers.
- **Layer A: Statistical Anomaly:** Employs Isolation Forest (and LOF fallback) to calculate spatial distribution outlier scores [0.0 - 1.0].
- **Layer B: Label Consistency:** Employs k-NN topological neighborhood analysis (k=11). Detects samples claiming to be benign when 90%+ of their geometric neighbors are malicious attacks.
- **Layer C: Model Loss Disparity:** Measures prediction margin and confidence loss under the currently active production IDS model.
- **Decision Combiner & Quarantine:** Combines layers into an explainable decision (Accepted, Quarantined, Review). Suspicious records are persisted in SQLite (quarantine.db) with full forensic reasons—zero silent drops.
- **Trusted Validation Gate:** Evaluates retrained candidate models against the isolated trusted_validation.csv holdout set. If performance drops by >3%, promotion is strictly BLOCKED.
- **Model Registry & Rollback:** Provides semantic versioning (v1.0.0, v1.1.0, etc.), lineage tracking, and instant 1-click fallback to the prior approved model binary.

- **REST API & SecOps Dashboard:** FastAPI server running on http://127.0.0.1:8000 with a glassmorphic dashboard featuring live telemetry, radar charts, quarantine triage tables, and model governance controls.

3. Industry 5.0 Autonomous Two-Agent Architecture

To bridge mathematical ML defense with real-world enterprise operations, the platform implements an **Industry 5.0 Human-in-the-Loop (HITL)** agentic paradigm. LLMs are NOT used for raw numeric packet scoring (avoiding latency, cost, and prompt injection); instead, high-speed ML conducts inline detection (<35 ms) while two specialized agents assist the human analyst:

Agent 1: SecOps Triage Agent (src/agents/triage_agent.py)

- **Trigger & Role:** Automatically inspects samples routed to the Quarantine Store.
  - **Explainability Engine:** Calculates statistical Z-score deviations and ranks root-cause anomaly features.
  - **Forensic Briefing:** Generates executive diagnosis, threat severity (CRITICAL / HIGH), confidence score (e.g. 98%), and clear recommended action (CONFIRM_POISON vs APPROVE_CLEAN).
  - **Analyst Action:** Analyst verifies the briefing on Tab 5 and applies action with one click.

Agent 2: Poisoning Guard Agent (src/agents/guard_agent.py)

- **Trigger & Role:** Orchestrates safe model retraining cycles and enforces model governance.
  - **Buffer Hygiene:** Verifies zero quarantined records exist in the candidate training buffer.
  - **Holdout Validation:** Trains candidate model and benchmarks against the isolated Trusted Holdout.
  - **Human-in-the-Loop Sign-Off:** Creates a formal **Model Promotion Ticket**. The candidate model is NEVER deployed automatically; it requires explicit analyst authorization on the dashboard to promote to production.

4. Measured Empirical Evaluation Benchmark

The following real experimental results were measured using experiments/run_experiment.py across all poisoning scenarios, proving the effectiveness of the multi-layer defense and validation gate:

Scenario	Rate	Defense Mode	Prec.	Recall	F1 Det.	FQR	Retrained IDS F1	Gate Action	Latency
Clean	0%	No Defense	1.00	1.00	1.00	0.00	1.0000	PROMOTED	33 ms
Label Flip	5%	No Defense	0.00	0.00	0.00	0.00	0.9882	PROMOTED	—
Label Flip	5%	Layer A (Stat)	0.11	0.07	0.08	0.03	0.9813	PROMOTED	54 ms
Label Flip	5%	Combined Def	1.00	1.00	1.00	0.00	1.0000	PROMOTED	54 ms
Label Flip	10%	No Defense	0.00	0.00	0.00	0.00	0.6486	PROMOTED (Compromised)	—
Label Flip	10%	Layer A (Stat)	0.22	0.07	0.10	0.03	0.6456	BLOCKED_BY_GATE	34 ms
Label Flip	10%	Combined Def	1.00	1.00	1.00	0.00	1.0000	PROMOTED	34 ms
Label Flip	20%	No Defense	0.00	0.00	0.00	0.00	0.0000	PROMOTED (Crashed)	—
Label Flip	20%	Combined Def	1.00	1.00	1.00	0.00	0.0000	BLOCKED_BY_GATE	29 ms
Sample Inj	10%	No Defense	0.00	0.00	0.00	0.00	1.0000	PROMOTED	—
Sample Inj	10%	Combined Def	1.00	1.00	1.00	0.00	1.0000	PROMOTED	35 ms
Sample Inj	20%	Combined Def	1.00	1.00	1.00	0.00	1.0000	PROMOTED	38 ms

5. Current Project State & Where We Paused

Current Status: The platform is fully operational, thoroughly tested, and running locally.

- **Git Repository:** Branch main is fully synchronized and tagged as v2.0.0.
- **Automated Tests:** 17 out of 17 test suites pass cleanly across unit, integration, and agent modules (pytest -v).
- **Active Web Dashboard:** Serving live on http://127.0.0.1:8000/ via FastAPI.
- **Buttons Fixed:** Triage action buttons on Tab 5 now provide instant confirmation alerts, auto-refresh the pending sample list, and sync with the Quarantine Store.
- **Model Registry:** Active production model is v1.0.0 (with capability to promote candidates to v1.1.0+ upon analyst sign-off).

6. Next-Steps Roadmap (Aage Kya Karna Hai)

Here is the precise step-by-step roadmap for the upcoming project phases:

Step 1: Supervisor Checkpoint Review: Demonstrate the live system to Sir Arfan Naich using the 7-step demo_runner.py CLI and the Web Dashboard. Show: (1) Clean vs Poisoned batch ingestion, (2) Layer B k-NN catching stealth label-flipping, (3) Triage Agent explainability cards, and (4) Poisoning Guard blocking degraded models and generating promotion tickets.

Step 2: Thesis & Academic Paper Updates: Update docs/final_thesis.md and docs/ieee_conference_paper.md to replace legacy ransomware sections with this new AML Poisoning Defense & Industry 5.0 Agentic Architecture. Insert the empirical benchmark table from Section 4.

Step 3: Optional Cloud Telemetry Connector: As a stretch goal, connect the stream replayer to live cloud VPC flow logs (AWS VPC Flow Logs or GCP Cloud Logging) via a lightweight webhook or Kafka/Redis broker.

Step 4: Multi-Seed Statistical Validation: Run experiments across 3 additional random seeds (42, 123, 999) to compute confidence intervals and standard deviations for the final thesis graphs.

7. Quick Command Cheat-Sheet

```
# 1. Run the interactive 7-step CLI demonstration:
python demo_runner.py

# 2. Launch FastAPI backend & SecOps Web Dashboard (http://127.0.0.1:8000):
uvicorn src.api.app:app --host 127.0.0.1 --port 8000 --reload

# 3. Execute all 17 automated unit and integration tests:
pytest -v

# 4. Re-calculate the full experimental benchmark matrix:
python -m experiments.run_experiment

# 5. Check Git release tags and branch status:
git tag -l
git status
```